# Obelisk Proof Assistant — Reference Demonstration

Variant: Higher-Order Logic & Morse–Kelley Class Theory • Author: Georgy Dunaev • contact@obelisk.mozmail.com

## 1) Propositional logic

**1.01) Axm. A01 (Weakening / Positive Implicative Axiom; K-Axiom; Russian: Утверждение следствия):**

$(\alpha \to (\beta \to \alpha))$

Typing judgment: $((\alpha : \mathrm{Bool}) \land (\beta : \mathrm{Bool}))$

**1.02) Axm. A02 (Distribution of Implication):**

$((\alpha \to (\beta \to \gamma)) \to ((\alpha \to \beta) \to (\alpha \to \gamma)))$

Typing judgment: $((\alpha : \mathrm{Bool}) \land ((\beta : \mathrm{Bool}) \land (\gamma : \mathrm{Bool})))$

**1.03) Axm. A03 (Conjunction Elimination Left):**

$((\alpha \land \beta) \to \alpha)$

Typing judgment: $((\alpha : \mathrm{Bool}) \land (\beta : \mathrm{Bool}))$

**1.04) Axm. A04 (Conjunction Elimination Right):**

$((\alpha \land \beta) \to \beta)$

Typing judgment: $((\alpha : \mathrm{Bool}) \land (\beta : \mathrm{Bool}))$

**1.05) Axm. A05 (Conjunction Introduction):**

$(\alpha \to (\beta \to (\alpha \land \beta)))$

Typing judgment: $((\alpha : \mathrm{Bool}) \land (\beta : \mathrm{Bool}))$

**1.06) Axm. A06 (Disjunction Introduction Left):**

$(\alpha \to (\alpha \lor \beta))$

Typing judgment: $((\alpha : \mathrm{Bool}) \land (\beta : \mathrm{Bool}))$

**1.07) Axm. A07 (Disjunction Introduction Right):**

$(\beta \to (\alpha \lor \beta))$

Typing judgment: $((\alpha : \mathrm{Bool}) \land (\beta : \mathrm{Bool}))$

**1.08) Axm. A08 (Disjunction Elimination / Cases):**

$((\alpha \to \gamma) \to ((\beta \to \gamma) \to ((\alpha \lor \beta) \to \gamma)))$

Typing judgment: $((\alpha : \mathrm{Bool}) \land ((\beta : \mathrm{Bool}) \land (\gamma : \mathrm{Bool})))$

**1.09) Axm. A09 (Negation Elimination / Contradiction to Bottom):**

$((\neg\alpha) \to (\alpha \to \bot))$

Typing judgment: $(\alpha : \mathrm{Bool})$

**1.10) Axm. A10 (Reductio Ad Absurdum / Negation Introduction):**

$((\alpha \to \beta) \to ((\alpha \to \neg\beta) \to \neg\alpha))$

Typing judgment: $((\alpha : \mathrm{Bool}) \land (\beta : \mathrm{Bool}))$

**1.11) Axm. A11 (Ex Falso Quodlibet):**

$(\bot \to \alpha)$

Typing judgment: $(\alpha : \mathrm{Bool})$

**1.12) Axm. A12 (Law of Excluded Middle):**

$(\alpha \lor \neg\alpha)$

Typing judgment: $(\alpha : \mathrm{Bool})$

**1.13) Rule Modus Ponens (MP):**

$\alpha, (\alpha \to \beta) \vdash \beta$

Typing judgment: $((\alpha : \mathrm{Bool}) \land (\beta : \mathrm{Bool}))$

## 2) Predicate logic

**2.01) Axm. Discourse domain is non-empty:**

$\exists(K : \mathrm{Class})\ \top$

Typing judgment: $(K : \mathrm{Class})$

**2.02) Rule Forall Instantiation (FI):**

$((\forall(x : \mathrm{Class})\ \varphi) \to \varphi[t/x])$

Syntactic side condition: $\mathrm{FFI}(t, v_x, \varphi)$

**2.03) Rule Exists Instantiation (EI):**

$(\varphi[t/x] \to \exists(x : \mathrm{Class})\ \varphi)$

Syntactic side condition: $\mathrm{FFI}(t, v_x, \varphi)$

**2.04) Rule Forall Abstraction (FA / Bernays):**

$(\psi \to \varphi) \vdash (\psi \to \forall(x : \mathrm{Class})\ \varphi)$

Syntactic side condition: $(\mathrm{NFV}(v_x, \psi) \land \mathrm{NFV}(v_x, \Gamma))$

**2.05) Rule Exists Abstraction (EA / Bernays):**

$(\varphi \to \psi) \vdash (\exists(x : \mathrm{Class})\ (\varphi \to \psi))$

Syntactic side condition: $(\mathrm{NFV}(v_x, \psi) \land \mathrm{NFV}(v_x, \Gamma))$

**2.06) Rule Generalisation (Gen):**

$\varphi \vdash \forall(x : \mathrm{Class})\ \varphi$

Syntactic side condition: $\mathrm{NFV}(v_x, \Gamma)$

## 3) Set/class-theoretic foundations of mathematics

**3.01) Axm. Class Comprehension Axiom:**

$\forall(x : \mathrm{Class})\ ((x \in \{\ y \mid \varphi(y)\ \}) \longleftrightarrow (\mathrm{isSet}(x) \land \varphi(x)))$

Syntactic side condition: $\mathrm{NFV}(v_C, \varphi)$

**3.02) Axm. Extensionality Axiom:**

$\forall(A, B : \mathrm{Class})\ ((\forall(x : \mathrm{Class})\ ((x \in A) \longleftrightarrow (x \in B))) \to (A = B))$

Typing judgment: $((A : \mathrm{Class}) \land ((B : \mathrm{Class}) \land (x : \mathrm{Class})))$

**3.08) Axm. Subset-Sethood / Heredity Axiom (Subclasses of Sets are Sets):**

$\forall(A, b : \mathrm{Class})\ ((\mathrm{isSet}(b) \land (A \subseteq b)) \to \mathrm{isSet}(A))$

Typing judgment: $((A : \mathrm{Class}) \land (b : \mathrm{Class}))$

**3.12) Axm. Regularity (Foundation / Cumulative □-Induction) Axiom:**

$((\forall(X : \mathrm{Class})\ ((\square(X) \subseteq X) \to (V \subseteq X))) \land (\forall(A : \mathrm{Class})\ ((A \neq \varnothing) \to \exists(x : \mathrm{Class})\ ((x \in A) \land ((x \cap A) = \varnothing)))))$

Typing judgment: $((X : \mathrm{Class}) \land ((A : \mathrm{Class}) \land (x : \mathrm{Class})))$

**3.13) Axm. Unordered Pair Definition & Pairing Axiom:**

$\forall(a, b : \mathrm{Class})\ ((\{a, b\} = \{\ x \mid ((x = a) \lor (x = b))\ \}) \land ((\mathrm{isSet}(a) \land \mathrm{isSet}(b)) \to \mathrm{isSet}(\{a, b\})))$

Typing judgment: $((a : \mathrm{Class}) \land ((b : \mathrm{Class}) \land (x : \mathrm{Class})))$

**3.14) Axm. Unary Union Definition & Union Axiom:**

$\forall(a : \mathrm{Class})\ ((\bigcup a = \{\ x \mid \exists(y : \mathrm{Class})\ ((y \in a) \land (x \in y))\ \}) \land (\mathrm{isSet}(a) \to \mathrm{isSet}(\bigcup a)))$

Typing judgment: $((a : \mathrm{Class}) \land ((x : \mathrm{Class}) \land (y : \mathrm{Class})))$

**3.22) Def. IF-THEN-ELSE-FI (Class conditional with NFV side condition and context turnstile):**

$W, \Gamma \vdash (⟦\mathrm{IF}\ \psi\ \mathrm{THEN}\ A\ \mathrm{ELSE}\ B\ \mathrm{FI}⟧ = \{\ v \mid ((\psi \land (v \in A)) \lor (\neg\psi \land (v \in B)))\ \})$

Syntactic side condition: $(\mathrm{NFV}(v_v, \psi) \land \mathrm{NFV}(v_v, \Gamma))$

**3.23) Def. simple_if (metalevel conditional over Type_Expression):**

$\forall(T_e : \mathrm{Type_Expression}),\ (\mathrm{simple_if}(⟦T_e⟧) : (\mathrm{Bool} \to (⟦T_e⟧ \to (⟦T_e⟧ \to ⟦T_e⟧))))$

$\forall(T_e : \mathrm{Type_Expression}),\ \forall(t_1 : ⟦T_e⟧)\ \forall(t_2 : ⟦T_e⟧)\ ((\mathrm{simple_if}(⟦T_e⟧, \mathrm{true}, t_1, t_2) = t_1) \land (\mathrm{simple_if}(⟦T_e⟧, \mathrm{false}, t_1, t_2) = t_2))$

Typing judgment: $((T_e : \mathrm{Type_Expression}) \land ((t_1 : ⟦T_e⟧) \land (t_2 : ⟦T_e⟧)))$

**3.24) Def. if_dep (dependent conditional over Bool → Type_Expression):**

$\forall(T_e : (\mathrm{Bool} \to \mathrm{Type_Expression})),\ (\mathrm{if_dep} : \Pi(b : \mathrm{Bool}),\ ⟦T_e(\mathrm{true})⟧ \to (⟦T_e(\mathrm{false})⟧ \to ⟦T_e(b)⟧))$

$\forall(T_e : (\mathrm{Bool} \to \mathrm{Type_Expression})),\ \forall(b : \mathrm{Bool})\ \forall(t_1 : ⟦T_e(\mathrm{true})⟧)\ \forall(t_2 : ⟦T_e(\mathrm{false})⟧)\ ((\mathrm{if_dep}(b, t_1, t_2) : ⟦T_e(b)⟧) \land ((\mathrm{if_dep}(\mathrm{true}, t_1, t_2) = t_1) \land (\mathrm{if_dep}(\mathrm{false}, t_1, t_2) = t_2)))$

Typing judgment: $((T_e : (\mathrm{Bool} \to \mathrm{Type_Expression})) \land ((b : \mathrm{Bool}) \land ((t_1 : ⟦T_e(\mathrm{true})⟧) \land (t_2 : ⟦T_e(\mathrm{false})⟧))))$

**3.32) Thm. Ordinal / Transitive-Class ∈-Induction (∈-Ind):**

$((\forall(K, A : \mathrm{Class})\ ((\mathrm{Tr}(K) \land ((K \cap \square(A)) \subseteq A)) \to (K \subseteq A))) \land ((\forall(x : \mathrm{Class})\ ((x \in \mathrm{On}) \to ((\forall(y : \mathrm{Class})\ ((y \in x) \to \varphi(y))) \to \varphi(x)))) \to \forall(x : \mathrm{Class})\ ((x \in \mathrm{On}) \to \varphi(x))))$

Syntactic side condition: $(\mathrm{Fresh}(v_y, v_x) \land \mathrm{NFV}(v_y, \varphi(x)))$

**3.33) Def. Power Class Definition (□(A)):**

$\forall(A, x : \mathrm{Class})\ ((x \in \square(A)) \longleftrightarrow (\mathrm{isSet}(x) \land (x \subseteq A)))$

Syntactic side condition: $\mathrm{Fresh}(v_x, v_A)$

**3.34) Axm. Power Set / Exponent Axiom (Power Class of a Set is a Set):**

∀(a : Class) (isSet(a) → isSet(□(a)))

Typing judgment: (a : Class)

**3.35) Def. Size Comparison Relation (A ≤ B / lesseq):**

∀(A : Class) ∀(B : Class) ((A ≤ B) ⟷ ∃(f : Class) (((isClassFunction(f) ∧ (A ⊆ Dom(f))) ∧ (f[A] ⊆ B)) ∧ isInjective(f)))

Typing judgment: ((A : Class) ∧ ((B : Class) ∧ (f : Class)))

**3.36) Def. Class Cardinality (|K|):**

∀(K : Class) (|K| = (On ∩ ⋂({ a | ((a ∈ On) ∧ (K ≤ a)) })))

Typing judgment: ((K : Class) ∧ (a : Class))

**3.37) Axm. Image / Replacement Axiom (isSet(F[a])):**

∀(F, a : Class) ((isClassFunction(F) ∧ isSet(a)) → isSet(F[a]))

Typing judgment: ((F : Class) ∧ (a : Class))

**3.38) Def. Strictly Smaller Size Relation (A ≺ B):**

∀(A : Class) ∀(B : Class) ((A ≺ B) ⟷ ¬(B ≤ A))

Typing judgment: ((A : Class) ∧ (B : Class))

**3.39) Axm. Limitation of Size Axiom ((X ∈ V) ⟷ (|X| ≺ |V|)):**

∀(X : Class) ((X ∈ V) ⟷ (|X| ≺ |V|))

Typing judgment: (X : Class)

**3.40) Def. Dunaev Tagged Ordered Pair:**

∀(A, B : Class) ([A, B]_D = { z | ∃(a ∈ A) (z = {{{{a}}}, {∅}}) ∨ ∃(b ∈ B) (z = {{{{b}}}, {∅, {∅}}}) })

Typing judgment: ((A : Class) ∧ (B : Class))

**3.41) Thm. First Projection of Dunaev Tagged Ordered Pair:**

∀(A, B : Class) ({ a | ({{{{{a}}}, {∅}} ∈ [A, B]_D) } = A)

Typing judgment: ((A : Class) ∧ (B : Class))

**3.42) Thm. Second Projection of Dunaev Tagged Ordered Pair:**

∀(A, B : Class) ({ b | ({{{{{b}}}, {∅, {∅}}} ∈ [A, B]_D) } = B)

Typing judgment: ((A : Class) ∧ (B : Class))

**3.45) Axm. Infinity Axiom:**

∃(x : Class) (isSet(x) ∧ ((∅ ∈ x) ∧ ∀(y : Class) ((y ∈ x) → ((y ∪ {y}) ∈ x))))

Typing judgment: ((x : Class) ∧ (y : Class))

**3.46) Axm. Global Choice Axiom:**

∃(F : Class) (isClassFunction(F) ∧ ∀(x : Class) ((isSet(x) ∧ (x ≠ ∅)) → (F(x) ∈ x)))

Typing judgment: ((F : Class) ∧ (x : Class))

**3.47) Def. Universal Class (V):**

(V = { x | (x = x) })

Typing judgment: (V : Class)

**3.53) Def. Transitive Class Predicate:**

(Tr(A) ⟷ ∀(x : Class) ((x ∈ A) → (x ⊆ A)))

Typing judgment: ((A : Class) ∧ (x : Class))

**3.66) Axm. Tarski's Axiom of Universes:**

∀(x : Class) (isSet(x) → ∃(u : Class) ((isUniverse(u) ∧ (x ∈ u)) ∧ isSet(u)))

Typing judgment: ((x : Class) ∧ (u : Class))

**3.67) Def. Semi-Singleton Class Predicate (semi_singleton):**

(semi_singleton(s) ⟷ ∀(x : Class) ∀(y : Class) (((x ∈ s) ∧ (y ∈ s)) → (x = y)))

Typing judgment: ((s : Class) ∧ ((x : Class) ∧ (y : Class)))

**3.68) Def. Semi-Singleton Subset Relation (s sss A):**

∀(s : Class) ∀(A : Class) ((s sss A) ⟷ ((s ⊆ A) ∧ ∀(x : Class) ∀(y : Class) (((x ∈ s) ∧ (y ∈ s)) → (x = y))))

Typing judgment: ((s : Class) ∧ ((A : Class) ∧ ((x : Class) ∧ (y : Class))))

**3.69) Thm. Theorem of Evaluation of a Family (F(x) = ⋃{ s | [x, s]_D ∈ F } and (Λx, t)(z) = t[z/x]):**

∀(F : Class) ∀(x : Class) (isFamily(F) → (F(x) = ⋃({ s | ([x, s]_D ∈ F) })))

Typing judgment: ((F : Class) ∧ ((x : Class) ∧ ((z : Class) ∧ ((s : Class) ∧ ((t : Class.Expression) ∧ ((v_x : Variable) ∧ (v_s : Variable)))))))

Syntactic side condition: (FFI(z, v_x, (s sss t)) ∧ (Fresh(v_s, v_x) ∧ NFV(v_s, t)))

**3.70) Def. Set-Theoretic Model of Dependent Sum (ΣB / Sigma Model):**

∀(B : Class) ∀(x : Class) ∀(y : Class) (isFamily(B) → (((x ∈ Dom(B)) ∧ (y ∈ B(x))) ⟷ ([x, y]_D ∈ { s | ∃(a : Class) ∃(b : Class) ((s = [a, b]_D) ∧ ((a ∈ Dom(B)) ∧ (b ∈ B(a)))) })))

Typing judgment: ((B : Class) ∧ ((x : Class) ∧ ((y : Class) ∧ ((s : Class) ∧ ((a : Class) ∧ (b : Class))))))

**3.71) Def. Set-Theoretic Model of Dependent Product (ΠB / Pi Predicate & Set-Section Models):**

∀(B : Class) ∀(f : Class) ((isFamily(B) ∧ (isFamily(f) ∧ (Dom(f) = Dom(B)))) → ∀(x : Class) ((x ∈ Dom(B)) → (f(x) ∈ B(x))))

Typing judgment: ((B : Class) ∧ ((f : Class) ∧ (x : Class)))

## 4) Metalogical laws

**4.01) Axm. Instantiation Axiom:**

∀(T_e : Type_Expression), ∀(G : Ctx) ∀(φ : Fm) ∀(t : ⟦T_e⟧) ∀(v : Variable) ((W, G ⊢ (∀(v : ⟦T_e⟧) φ)) → (W, G ⊢ (let v = t in φ)))

Typing judgment: ((T_e : Type_Expression) ∧ ((t : ⟦T_e⟧) ∧ (v : Variable)))

**4.02) Axm. Generalisation Axiom:**

∀(T_e : Type_Expression), ∀(Γ : Ctx) ∀(φ : Fm) ∀(v : Variable) (¬(Enc(v) ∈ Ctx.FV(Γ)) → ((W, Γ ⊢ φ) → (W, Γ ⊢ ∀(v : ⟦T_e⟧) φ)))

Typing judgment: ((T_e : Type_Expression) ∧ ((Γ : Ctx) ∧ ((φ : Fm) ∧ (v : Variable))))

Syntactic side condition: NFV(v, Γ)

**4.03) Rule CoNecessitation:**

(F, [] ⊢ ‹F, Γ ⊢ φ›) ⊢ (F, Γ ⊢ φ)

Typing judgment: ((F : Theory) ∧ ((Γ : Ctx) ∧ (φ : Fm)))

**4.04) Def. Quantifier Unique (!):**

∀(φ : Fm) ((!(z : Class) φ) ⟷ ∀(x, y : Class) ((φ[x/z] ∧ φ[y/z]) → (x = y)))

Typing judgment: ((z : Variable) ∧ ((x : Variable) ∧ (y : Variable)))

**4.05) Def. Quantifier Exists and Unique (∃!):**

∀(φ : Fm) (T, [] ⊢ ((∃!(x : Class) φ) ⟷ ((∃(x : Class) φ) ∧ !(x : Class) φ)))

Typing judgment: ((x : Variable) ∧ (φ : Fm))

**4.06) Axm. Class Comprehension Axiom (Metatheory):**

∀(φ : Fm) ∀(x : Name) ∀(k : Class.Expr) (FFI(k, x, φ) → (T, [] ⊢ ((k ∈ { x | φ }) ⟷ (isSet(k) ∧ φ[k/x]))))

Syntactic side condition: FFI(k, x, φ)

**4.07) Axm. Completeness / Reflection & Soundness of Full Theory F:**

∀(φ : Fm) (((F, [] ⊢ φ) → ⟦φ⟧) ∧ (⟦φ⟧ → (F, [] ⊢ φ)))

Typing judgment: ((φ : Fm) ∧ (F : Theory))

**4.08) Axm. Theory Inclusion (Model(T) ⊆ Model(F)):**

∀(Γ : Ctx) ∀(φ : Fm) ((T, Γ ⊢ φ) → (F, Γ ⊢ φ))

Typing judgment: ((T : Theory) ∧ ((F : Theory) ∧ ((Γ : Ctx) ∧ (φ : Fm))))

**4.09) Def. Syntactic Encoding Operator (Enc):**

∀(T_e : Type_Expression), ∀(s : ⟦T_e⟧) (isSet(Enc(s)) ∧ (Enc(s) ∈ Emb(⟦T_e⟧)))

Typing judgment: ((T_e : Type_Expression) ∧ (s : ⟦T_e⟧))

**4.10) Def. Syntactic Decoding & Roundtrip Law (Dec):**

∀(T_e : Type_Expression), ∀(s : ⟦T_e⟧) ((Dec(Enc(s), Emb(⟦T_e⟧)) : ⟦T_e⟧) = (s : ⟦T_e⟧))

Typing judgment: ((T_e : Type_Expression) ∧ (s : ⟦T_e⟧))

**4.11) Def. Dual Type Embedding Operator (Emb):**

∀(X : Class) ((Emb(Class)(X) ⟷ ⊤) ∧ (Emb(Set)(X) ⟷ isSet(X)))

Typing judgment: (X : Class)

**4.12) Def. Proof-Object Class Family (ProofsOf):**

∀(s : Stmt) ∀(p : Prf) ((Enc(p) ∈ ProofsOf(Enc(s))) ⟷ CheckProof(p, s))

Typing judgment: ((s : Stmt) ∧ (p : Prf))

**4.13) Def. Class of Provable Statements (Theorems):**

∀(s : Stmt) ((Enc(s) ∈ Theorems) ⟷ (∃(p : Class) (p ∈ ProofsOf(Enc(s)))))

Typing judgment: ((s : Stmt) ∧ (p : Class))

## 5) Tactics and inference rules

**5.01) Thm. Elementary Theorems:**

((F, [(b ∈ X)] ⊢ ∃(X : Class) (b ∈ X)) ∧ (F, [(b ∈ X)] ⊢ isSet(b)))

Typing judgment: ((b : Class) ∧ (X : Class))

**5.02) Rule Definition to Introduction Rule:**

(F, [(b ∈ X)] ⊢ ∃(X : Class) (b ∈ X)) ⊢ (F, [(b ∈ X)] ⊢ isSet(b))

Typing judgment: ((b : Class) ∧ (X : Class))

**5.03) Rule Deduction Rule Application:**

(F, [] ⊢ ((b ∈ X) → isSet(b)))

Typing judgment: ((b : Class) ∧ (X : Class))

**5.04) Thm. Tactics Proof Example:**

F, [] ⊢ ((∃(Y : Class) ∀(x : Class) (x ∈ Y)) → ∀(x : Class) ∃(Y : Class) (x ∈ Y))

Typing judgment: ((x : Class) ∧ (Y : Class))

## 6) Incompleteness-related statements

**6.01) Axm. Theories (Metalevel Stratification):**

((Model(W) ⊆ Model(Q)) ∧ ((Model(Q) ⊆ Model(T)) ∧ (Model(T) ⊆ Model(F))))

Typing judgment: ((W : Theory) ∧ ((Q : Theory) ∧ ((T : Theory) ∧ (F : Theory))))

**6.02) Thm. Löb's Theorem:**

∀(φ : Fm) (Q, [] ⊢ ((T, [] ⊢ ((T, [] ⊢ φ) → φ)) → (T, [] ⊢ φ)))

Typing judgment: (φ : Fm)

**6.03) Thm. Gödel's Second Incompleteness Theorem:**

Q, [] ⊢ (¬(T, [] ⊢ ⊥) → ¬(T, [] ⊢ ¬(T, [] ⊢ ⊥)))

Typing judgment: ((T : Theory) ∧ (Q : Theory))

## Appendix: NBG Formulas-as-Types Standard Library

30 closed proof terms compiled by NBGSurfaceCompiler in Poly/ML. A formula is a type; a proof program is a term inhabiting it. The Dunaev pair injectivity theorem is the class-correct replacement for Kuratowski pairing on proper classes.

| Theorem | Formula (Type) | AST | Norm |
|---|---|---|---|
| empty_class_no_elements | (∀(z : Class) ((z ∈ { x \| ⊥ }) → ⊥)) | 7 | 7 |
| empty_class_is_subclass | (∀(X : Class) (∀(z : Class) ((z ∈ { x \| ⊥ }) → (z ∈ X)))) | 8 | 8 |
| universal_class_characterization | (∀(z : Class) (((z ∈ { x \| ⊤ }) → isSet(z)) ∧ (isSet(z) → (z ∈ { x \| ⊤ })))) | 15 | 15 |
| russell_class_is_not_a_set | (isSet({ x \| ((x ∈ x) → ⊥) }) → ⊥) | 19 | 32 |
| binary_inter_comm | (∀(A : Class) (∀(B : Class) (∀(z : Class) (((z ∈ { x \| ((x ∈ A) ∧ (x ∈ B)) }) → (z ∈ { x \| ((x ∈ B) ∧ (x ∈ A)) })) ∧ ((z ∈ { x \| ((x ∈ B) ∧ (x ∈ A)) }) → (z ∈ { x \| ((x ∈ A) ∧ (x ∈ B)) }))))))) | 50 | 50 |
| binary_union_comm | (∀(A : Class) (∀(B : Class) (∀(z : Class) (((z ∈ { x \| ((x ∈ A) ∨ (x ∈ B)) }) → (z ∈ { x \| ((x ∈ B) ∨ (x ∈ A)) })) ∧ ((z ∈ { x \| ((x ∈ B) ∨ (x ∈ A)) }) → (z ∈ { x \| ((x ∈ A) ∨ (x ∈ B)) }))))))) | 44 | 44 |
| class_diff_subclass_left | (∀(A : Class) (∀(B : Class) (∀(z : Class) ((z ∈ { x \| ((x ∈ A) ∧ ((x ∈ B) → ⊥)) }) → (z ∈ A))))) | 10 | 10 |
| class_eq_sym | (∀(X : Class) (∀(Y : Class) ((X = Y) → (Y = X)))) | 6 | 6 |
| class_eq_trans | (∀(X : Class) (∀(Y : Class) (∀(Z : Class) (((X = Y) ∧ (Y = Z)) → (X = Z))))) | 9 | 9 |
| quantifier_swap | (∀(R : Class) ((∃(y : Set) (∀(x : Set) ([x, y]_D ∈ R))) → (∀(x : Set) (∃(y : Set) ([x, y]_D ∈ R))))) | 8 | 8 |
| dunaev_tagged_pair_projections | (∀(A : Class) (∀(B : Class) ((π₁([A, B]_D) = A) ∧ (π₂([A, B]_D) = B)))) | 3 | 3 |
| cantor_diagonal_theorem | (∀(S : Class) (∀(R : Class) (∀(c : Class) ((c ∈ S) → ((∀(z : Class) (((z ∈ { x \| ((x ∈ S) ∧ (([x, x]_D ∈ R) → ⊥)) }) → ([c, z]_D ∈ R)) ∧ (([c, z]_D ∈ R) → (z ∈ { x \| ((x ∈ S) ∧ (([x, x]_D ∈ R) → ⊥)) }))))) → ⊥)))) | 36 | 67 |
| self_in_singleton | (∀(x : Class) (isSet(x) → (x ∈ { u \| (u = x) }))) | 8 | 8 |
| self_in_succ | (∀(x : Class) (isSet(x) → (x ∈ { u \| ((u ∈ x) ∨ (u = x)) }))) | 9 | 9 |
| class_sub_succ | (∀(x : Class) (∀(z : Class) ((z ∈ x) → (z ∈ { u \| ((u ∈ x) ∨ (u = x)) })))) | 12 | 12 |
| dunaev_pair_injective | (∀(A : Class) (∀(B : Class) (∀(C : Class) (∀(E : Class) (([A, B]_D = [C, E]_D) → ((A = C) ∧ (B = E))))))) | 26 | 26 |
| dunaev_cartesian_elim | (∀(A : Class) (∀(B : Class) (∀(p : Class) ((p ∈ { q \| (∃(u : Set) (∃(v : Set) (((u ∈ A) ∧ (v ∈ B)) ∧ (q = [u, v]_D)))) }) → (∃(u : Set) (∃(v : Set) (((u ∈ A) ∧ (v ∈ B)) ∧ (p = [u, v]_D)))))))) | 9 | 9 |
| inter_sub_left | (∀(A : Class) (∀(B : Class) (∀(z : Class) ((z ∈ { u \| ((u ∈ A) ∧ (u ∈ B)) }) → (z ∈ A))))) | 10 | 10 |
| inter_sub_right | (∀(A : Class) (∀(B : Class) (∀(z : Class) ((z ∈ { u \| ((u ∈ A) ∧ (u ∈ B)) }) → (z ∈ B))))) | 10 | 10 |
| union_sub_left | (∀(A : Class) (∀(B : Class) (∀(z : Class) ((z ∈ A) → (z ∈ { u \| ((u ∈ A) ∨ (u ∈ B)) }))))) | 13 | 13 |
| semi_singleton_sub_elim | (∀(s : Class) (∀(A : Class) (∀(x : Class) (∀(y : Class) (((∀(u : Class) ((u ∈ s) → (u ∈ A))) ∧ (∀(u : Class) (∀(v : Class) (((u ∈ s) ∧ (v ∈ s)) → (u = v))))) → (((x ∈ s) ∧ (y ∈ s)) → ((x ∈ A) ∧ (x = y)))))))) | 19 | 19 |
| family_fiber_char | (∀(A : Class) (∀(s : Class) ((s ∈ { u \| (u ⊆ A) }) → (isSet(s) ∧ (s ⊆ A))))) | 7 | 7 |
| sigma_dependent_sum_intro | (∀(B : Class) (∀(x : Set) (∀(y : Set) ((isSet([x, y]_D) ∧ ((x ∈ Dom(B)) ∧ (y ∈ ValOf(B, x)))) → ([x, y]_D ∈ { p \| (∃(a : Set) (∃(b : Set) (((a ∈ Dom(B)) ∧ (b ∈ ValOf(B, a))) ∧ (p = [a, b]_D)))) }))))) | 16 | 16 |
| pi_dependent_product_elim | (∀(B : Class) (∀(f : Class) (∀(x : Class) (((∀(u : Class) ((u ∈ Dom(B)) → (ValOf(f, u) ∈ ValOf(B, u)))) ∧ (x ∈ Dom(B))) → (ValOf(f, x) ∈ ValOf(B, x)))))) | 10 | 10 |
| union_sub_right | (∀(A : Class) (∀(B : Class) (∀(z : Class) ((z ∈ B) → (z ∈ { u \| ((u ∈ A) ∨ (u ∈ B)) }))))) | 13 | 13 |
| model_stratification_trans | (∀(W : Class) (∀(Q : Class) (∀(T : Class) (∀(F : Class) (((∀(z : Class) ((z ∈ Model(W)) → (z ∈ Model(Q)))) ∧ ((∀(z : Class) ((z ∈ Model(Q)) → (z ∈ Model(T)))) ∧ (∀(z : Class) ((z ∈ Model(T)) → (z ∈ Model(F)))))) → (∀(z : Class) ((z ∈ Model(W)) → (z ∈ Model(F)))))))))) | 22 | 22 |
| if_dep_true_elim | (∀(A : Class) (∀(T1 : Class) (∀(T2 : Class) (∀(x : Class) (∀(z : Class) (((((x ∈ A) → (if_dep(x, A, ValOf(T1, x), ValOf(T2, x)) = ValOf(T1, x))) ∧ (((x ∈ A) → ⊥) → (if_dep(x, A, ValOf(T1, x), ValOf(T2, x)) = ValOf(T2, x)))) ∧ ((x ∈ A) ∧ (z ∈ if_dep(x, A, ValOf(T1, x), ValOf(T2, x))))) → (z ∈ ValOf(T1, x)))))))) | 17 | 17 |
| if_dep_false_elim | (∀(A : Class) (∀(T1 : Class) (∀(T2 : Class) (∀(x : Class) (∀(z : Class) (((((x ∈ A) → (if_dep(x, A, ValOf(T1, x), ValOf(T2, x)) = ValOf(T1, x))) ∧ (((x ∈ A) → ⊥) → (if_dep(x, A, ValOf(T1, x), ValOf(T2, x)) = ValOf(T2, x)))) ∧ (((x ∈ A) → ⊥) ∧ (z ∈ if_dep(x, A, ValOf(T1, x), ValOf(T2, x))))) → (z ∈ ValOf(T2, x)))))))) | 17 | 17 |
| inter_idempotent_elim | (∀(A : Class) (∀(z : Class) ((z ∈ { u \| ((u ∈ A) ∧ (u ∈ A)) }) → (z ∈ A)))) | 9 | 9 |
| singleton_membership_eq | (∀(a : Class) (∀(z : Class) ((z ∈ { u \| (u = a) }) → (z = a)))) | 8 | 8 |

Reproduce locally: python3 obelisk_cli.py compile examples/nbg_verified_programs.nbg